Examen práctico — Práctica 2 (simulacro): Consultas sobre la lista

Duración máxima: 1 h 30 min. Sin Internet ni asistentes de IA.

Dispones de:

- Tu implementación de la práctica 2: la clase `Track` completa y la clase `Playlist` hasta el método `add` (constructor, constructor de copia, `operator=`, destructor, `size`, `empty` y `add`). **No necesitas nada más** de la práctica: las pruebas del examen no usan `get`, `findById`, `getCurrent`, `insertAt`, `removeById`, `load`, `merge`, `clear`, `next`, `prev`, `totalDuration` ni `operator<<`.
- Las 8 pruebas del grupo **X** (X11 a X18) correspondientes a este examen.
- Seminario de C++ en https://www.dlsi.ua.es/asignaturas/eda/seminario-c++-parte1.pdf
- Copia del manual de referencia de C++ en https://www.dlsi.ua.es/asignaturas/eda/cpp

Ejecuta `./run_tests.sh X` para ejecutar las pruebas del examen y ver tu calificación. Puedes ejecutar una sola con `./run_tests.sh X15`.

Ampliación: consultas sobre la lista

Se añaden a `Playlist` dos métodos. **Declara ambos en `Playlist.h`** (añade `#include <vector>`; no cambies ninguna firma de la práctica) e impleméntalos en `Playlist.cc`.

Los dos métodos son **independientes**: puedes hacerlos en el orden que quieras, y las pruebas de uno no necesitan el otro. Cada prueba pasada suma por igual, así que si te atascas en un método, pasa al otro.

Recuerda las reglas de la práctica: recorre la lista con los punteros de los nodos, no uses contenedores de la STL para guardar las pistas (el `vector<string>` que se usa aquí solo transporta ids) y no dejes fugas de memoria ni accesos inválidos (se comprueba con valgrind).

En todos los ejemplos se parte de esta lista (id, duración en segundos):

posición	0	1	2	3	4
`id`	`t01`	`t02`	`t03`	`t04`	`t05`
duración	300	100	200	50	400

```
Playlist p;
p.add(Track("t01", "Uno",    300));
p.add(Track("t02", "Dos",    100));
p.add(Track("t03", "Tres",   200));
p.add(Track("t04", "Cuatro",  50));
```

```
p.add(Track("t05", "Cinco",  400));
```

1. `vector<string> shorterThan(const string& id) const` — pruebas X11 a X14

Se toma como **referencia** la pista cuyo `id` es el indicado, y se devuelven los `id` de todas las pistas que duran **estrictamente menos** que ella, **en el mismo orden en que aparecen en la lista**.

- Las pistas de **igual duración** que la de referencia no se incluyen, ni tampoco la propia pista de referencia.
- Si no hay ninguna pista con ese `id` (inexistente, vacío, solo un prefijo, con otras mayúsculas...) o la lista está vacía, devuelve un vector vacío.
- Si ninguna pista es más corta que la de referencia, devuelve un vector vacío.
- No modifica la lista.

Ejemplos:

```
p.shorterThan("t03");   // {"t02", "t04"}   (100 y 50 son menores que 200)
p.shorterThan("t05");   // {"t01", "t02", "t03", "t04"}   (la más larga)
p.shorterThan("t01");   // {"t02", "t03", "t04"}
p.shorterThan("t04");   // {}   (la más corta: ninguna dura menos de 50)
p.shorterThan("t99");   // {}   (no existe)
p.shorterThan("");      // {}   (no existe)
p.shorterThan("T03");   // {}   (las mayúsculas cuentan)
```

Con empates (si `t06` durase también 200, `p.shorterThan("t03")` seguiría dando `{"t02", "t04"}`: `t06` no es *estrictamente* menor).

2. `Playlist tracksAfterLast(const vector<string>& ids) const` — pruebas X15 a X18

El vector `ids` contiene los `id` de unas **pistas marcadas**. Entre las pistas de la lista que están marcadas, se toma la **última** (la más cercana al final de la lista). El método devuelve una **lista nueva** con una copia de todas las pistas que van **después** de esa última marcada, en el mismo orden que tenían. La pista marcada **no** se incluye.

- El orden de los `id` dentro de `ids` no importa: lo que cuenta es la posición de las pistas **en la lista**.
- Si `ids` contiene ids repetidos o ids que no existen en la lista, esos ids se ignoran sin más.
- Si no hay ninguna pista marcada (`ids` vacío, o ningún id existe en la lista), devuelve una lista vacía.
- Si la última pista marcada es la última de la lista, no hay nada después: devuelve una lista vacía.
- La lista devuelta es una **copia independiente**: tiene sus propios nodos, y modificar una de las dos listas no afecta a la otra.

- No modifica la lista original.

Ejemplos:

```
vector<string> ids1 = {"t03"};
Playlist r1 = p.tracksAfterLast(ids1);   // contiene t04 y t05          ↪
(r1.size() == 2)

vector<string> ids2 = {"t01", "t04"};
Playlist r2 = p.tracksAfterLast(ids2);   // la última marcada es t04:  ↪
contiene solo t05

vector<string> ids3 = {"t04", "t01"};    // mismo resultado que ids2:  ↪
el orden de ids no importa
Playlist r3 = p.tracksAfterLast(ids3);   // contiene solo t05

vector<string> ids4 = {"t01"};
Playlist r4 = p.tracksAfterLast(ids4);   // contiene t02, t03, t04 y t05

vector<string> ids5 = {"t05", "t02"};
Playlist r5 = p.tracksAfterLast(ids5);   // la última marcada es t05:  ↪
lista vacía

vector<string> ids6 = {"zz", "t02", "t02"};
Playlist r6 = p.tracksAfterLast(ids6);   // zz no existe y t02 está    ↪
repetido: contiene t03, t04 y t05

vector<string> ninguno;
Playlist r7 = p.tracksAfterLast(ninguno);   // sin marcas: lista vacía
vector<string> inexistentes = {"t99"};
Playlist r8 = p.tracksAfterLast(inexistentes);   // ningún id existe:  ↪
lista vacía
// En todos los casos, p sigue teniendo 5 pistas.
```

> **Pista:** es más sencillo implementarlo **empezando el recorrido por el final** de la lista. Recorre hacia atrás desde `tail` usando `prev`: la primera pista que encuentres cuyo id esté en `ids` es la **última marcada** (si llegas al principio sin encontrar ninguna, devuelve la lista vacía). Desde esa pista, recorre hacia delante con `next` copiando cada pista con `add` en la lista resultado: así quedan en su orden original. Buscar desde el principio obliga a recorrer toda la lista y a recordar la última coincidencia.

Evaluación

- Hay 8 pruebas (X11 a X18) con el mismo peso: **nota = pruebas pasadas / 8 × 10**.
- Una prueba que pasa pero tiene errores de memoria (valgrind) no cuenta.
- El código debe compilar sin errores ni warnings.
- No se puede modificar la carpeta `tests/`.